**ĐẠI HỌC BÁCH KHOA HÀ NỘI**

## PROJECT I

# Phát hiện thư rác sử dụng thuật toán SVM

**NGUYỄN MẠNH THẮNG**
thang.nm235426@sis.hust.edu.vn

CHƯƠNG TRÌNH ĐÀO TẠO: KỸ THUẬT MÁY TÍNH-68

Giảng viên hướng dẫn: Phạm Mạnh Tuấn

Hà Nội, 1/2026

# Mục lục

**1 Giới thiệu bài toán** **4**
1.1 Bối cảnh và tầm quan trọng . . . . . 4
1.2 Phát biểu bài toán . . . . . 4
1.3 Phương pháp tiếp cận và Mục tiêu dự án . . . . . 4
1.3.1 Phương pháp tiếp cận . . . . . 4
1.3.2 Mục tiêu dự án . . . . . 5

**2 Cơ sở lý thuyết** **6**
2.1 Hard margin SVM . . . . . 6
2.1.1 Ý tưởng của Hard margin . . . . . 6
2.1.2 Mô hình toán học . . . . . 6
2.2 Soft margin SVM . . . . . 8
2.2.1 Ý tưởng của Soft margin . . . . . 8
2.2.2 Mô hình toán học . . . . . 8

**3 Các bước tiến hành** **10**
3.1 Dataset . . . . . 10
3.2 Quy trình xử lý dữ liệu thô . . . . . 10
3.2.1 Làm sạch dữ liệu (Data Cleaning) . . . . . 10
3.2.2 Tiền xử lý văn bản (Text Preprocessing) . . . . . 10
3.2.3 Trích xuất đặc trưng . . . . . 10
3.3 Các chỉ số đánh giá (Metrics) . . . . . 10
3.4 Triển khai thuật toán . . . . . 11
3.4.1 Công cụ và môi trường triển khai . . . . . 11
3.4.2 Chia tập dữ liệu . . . . . 11
3.4.3 Cài đặt mô hình các loại . . . . . 11

**4 Kết quả** **12**
4.1 Kết quả đối với từng thuật toán . . . . . 12
4.2 So sánh kết quả giữa các thuật toán . . . . . 16

# Danh sách hình vẽ


1 Phân tích bài toán SVM. . . . . 6
2 Confusion matrix đối với thuật toán SVM. . . . . 12
3 Confusion matrix đối với thuật toán Naive Bayes. . . . . 13
4 Confusion matrix đối với thuật toán Logistics Regression. . . . . 14
5 Confusion matrix đối với thuật toán kNN. . . . . 15
6 Biểu đồ so sánh kết quả giữa các thuật toán. . . . . 16

# 1 Giới thiệu bài toán

## 1.1 Bối cảnh và tầm quan trọng

Trong kỷ nguyên số hiện nay, các phương thức giao tiếp qua văn bản như Email, SMS hay tin nhắn trên mạng xã hội là phần không thể thiếu trong công việc và đời sống. Tuy nhiên, đi kèm với sự tiện lợi đó là sự bùng nổ của thư rác (Spam).

Thư rác không chỉ gây phiền nhiễu, làm giảm năng suất làm việc của người dùng mà còn tiềm ẩn nhiều nguy cơ về an ninh mạng như:

- **Lừa đảo (Phishing):** Đánh cắp thông tin cá nhân và tài khoản
- **Mã độc (Malware):** Phát tán tệp tin độc hại gây hư hại hệ thống
- **Chiếm dụng tài nguyên:** Tiêu tốn băng thông và dung lượng lưu trữ máy chủ

Vì vậy, việc phát triển một hệ thống phân loại thư rác tự động, chính xác và hiệu quả là một nhu cầu cấp thiết đối với các tổ chức và cá nhân.

## 1.2 Phát biểu bài toán

Về mặt kỹ thuật, phát hiện thư rác là một bài toán **Phân loại nhị phân** (Binary Classification) trong lĩnh vực Học máy (Machine Learning) và Xử lý ngôn ngữ tự nhiên (NLP).

Dữ liệu đầu vào thường là các chuỗi văn bản không cấu trúc. Nhiệm vụ của mô hình là gán nhãn cho mỗi thông điệp vào một trong hai lớp:

- **Ham (Thư thường):** Nội dung hợp lệ, có ích
- **Spam (Thư rác):** Nội dung quảng cáo không mong muốn hoặc độc hại

## 1.3 Phương pháp tiếp cận và Mục tiêu dự án

### 1.3.1 Phương pháp tiếp cận

Đồ án tập trung triển khai thuật toán **Support Vector Machine (SVM)** – một trong những thuật toán mạnh mẽ và hiệu quả cho các bài toán phân loại văn bản nhờ khả năng xử lý dữ liệu trong không gian nhiều chiều.

Để có cái nhìn khách quan và đánh giá toàn diện về hiệu quả của mô hình, dự án cũng thực hiện triển khai so sánh với các thuật toán học máy phổ biến khác bao gồm:

- **Naive Bayes:** Dựa trên định lý xác suất Bayes, thường hiệu quả với dữ liệu văn bản.
- **k-Nearest Neighbors (kNN):** Thuật toán dựa trên khoảng cách giữa các điểm dữ liệu.
- **Logistic Regression:** Mô hình phân loại thống kê cơ bản nhưng mang lại hiệu suất ổn định.

### 1.3.2 Mục tiêu dự án

1. Tiền xử lý dữ liệu (Cleaning, Tokenization, Vectorization).
2. Xây dựng và huấn luyện mô hình bằng SVM và các thuật toán đối chứng.
3. Đánh giá dựa trên Accuracy, Precision, Recall và F1-score.

# 2 Cơ sở lý thuyết

## 2.1 Hard margin SVM

### 2.1.1 Ý tưởng của Hard margin

Hard Margin SVM là phiên bản cơ bản nhất của thuật toán, được áp dụng khi dữ liệu của hai lớp là **phân tách tuyến tính hoàn toàn** (linearly separable), nghĩa là tồn tại một siêu phẳng có thể chia tuyệt đối các điểm dữ liệu mà không có sai sót nào.

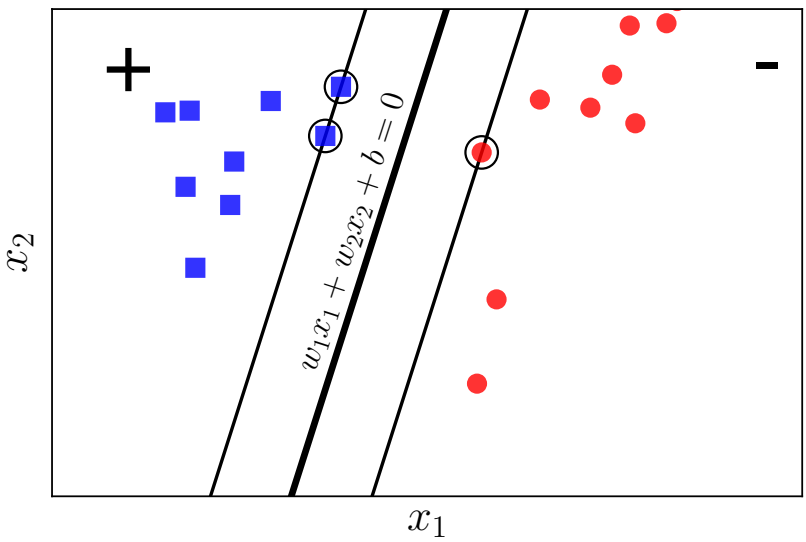
Ý tưởng của Hard Margin SVM là tìm siêu phẳng phân loại sao cho nó cách đều cả hai lớp với một khoảng cách **lớn nhất**. Tức là tất cả các điểm thuộc cùng một lớp sẽ nằm về một phía và không có điểm nào nằm ở bên trong lề (margin).

### 2.1.2 Mô hình toán học

Giả sử rằng các cặp dữ liệu của *training set* là

$$(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_N, y_N)$$

với vector $\mathbf{x}_i \in \mathbb{R}^d$ thể hiện đầu vào của một điểm dữ liệu và $y_i$ là nhãn của điểm dữ liệu đó. $d$ là số chiều của dữ liệu và $N$ là số điểm dữ liệu. Giả sử rằng nhãn của mỗi điểm dữ liệu được xác định bởi $y_i = 1$ (class xanh) hoặc $y_i = -1$ (class đỏ), tức là các điểm màu xanh sẽ ở phía dương của *hyperplane*, các điểm màu đỏ sẽ nằm ở phía âm (đây là giả sử quan trọng để có thể giải được thuật toán SVM, sẽ có giải thích ở phía dưới).




Hình 1: Phân tích bài toán SVM.

Ta quan sát thấy một điểm quan trọng sau đây: với cặp dữ liệu $(\mathbf{x}_n, y_n)$ bất kỳ, khoảng cách từ điểm đó tới *hyperplane* phân loại là:

$$\frac{y_n(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b)}{\|\mathbf{w}\|_2}$$

Điều này có thể dễ nhận thấy vì theo giả sử ở trên, $y_n$ luôn cùng dấu với phía của $\mathbf{x}_n$. Từ đó suy ra $y_n$ cùng dấu với $(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b)$, và tử số luôn là một số không âm.

Vì *margin* được tính là khoảng cách gần nhất từ 1 điểm tới mặt đó (bất kể điểm nào trong hai *classes*):

$$\text{margin} = \min_n \frac{y_n(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b)}{\|\mathbf{w}\|_2} \tag{1}$$

Bài toán tối ưu trong SVM chính là bài toán tìm $\mathbf{w}$ và $b$ sao cho *margin* này đạt giá trị lớn nhất:

$$\begin{aligned} (\mathbf{w}, b) &= \arg\max_{\mathbf{w},b} \left\{ \min_n \frac{y_n(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b)}{\|\mathbf{w}\|_2} \right\} = \arg\max_{\mathbf{w},b} \left\{ \frac{1}{\|\mathbf{w}\|_2} \min_n y_n(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b) \right\} \\ \text{vđk:} \quad & y_n(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b) > 0, \quad \forall n = 1, 2, \ldots, N \end{aligned} \tag{2}$$

Nhìn vào phương trình (2), ta chú ý thấy điều kiện $y_n(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b) > 0, \quad \forall n = 1, 2, \ldots, N$, điều kiện này giúp giới hạn không gian tìm kiếm để đảm bảo sao cho các giá trị $\mathbf{w}, b$ thoả mãn luôn cho ta một mặt phẳng phân chia rời rạc 2 tập điểm, tức là điểm xanh sẽ ở phía dương, điểm đỏ sẽ ở phía âm. Ví dụ cụ thể:

$$\begin{aligned} y_i &= 1(\text{điểm xanh}) \Rightarrow (\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b) > 0 \\ y_i &= -1(\text{điểm đỏ}) \Rightarrow (\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b) < 0 \end{aligned}$$

như vậy, khi hợp các điều kiện này lại với nhau ta sẽ thu được không gian các giá trị $\mathbf{w}, b$ thoả mãn. Đây chính là lý do tại sao giả sử phía trên lại quan trọng.

Để ý một chút, ta có thể thấy, ý tưởng của thuật toán SVM ta vừa nêu ở trên là: "*tìm siêu phẳng phân loại sao cho nó cách đều cả hai lớp với một khoảng cách* ***lớn nhất***", nhưng ở bài toán tối ưu trong SVM (2) thì lại đi *maximum* giá trị của *margin*. Một cách trực quan, ta chưa thấy rõ được sự liên kết giữa bài toán tối ưu (2) với ý tưởng ban đầu, nhưng thực ra bài toán tối ưu (2) với mục tiêu ban đầu lại hoàn toàn giống nhau. Điều này sẽ được thể hiện qua một chứng minh phản chứng, sau đây.

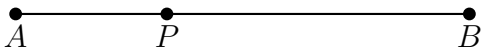
**Chứng minh:**

+) Giả định, ta đã giải quyết được bài toán tối ưu (2).

+) Gọi $\mathbf{m}_1^{\mathrm{T}}\mathbf{x} + m_2 = 0$ là một *hyperplane* cho kết quả tối ưu; $d_1, d_2$ lần lượt là khoảng cách gần nhất từ tập xanh, tập đỏ đến $\mathbf{m}_1^{\mathrm{T}}\mathbf{x} + m_2 = 0$.

+) Giả sử: $d_1 \neq d_2$, không mất tính tổng quát ta có thể coi $d_1 < d_2 \Rightarrow \text{margin} = d_1$.

+) Minh hoạ $d_1, d_2$ trên trên hình vẽ, với $AP = d_1, BP = d_2$.




+) Nhìn vào hình vẽ, ta thấy ta hoàn toàn có thể thu được giá trị lớn hơn của $d_1$ bằng cách dịch điểm $P$ về phía chính giữa đoạn thẳng, hay tức là ta có thể thu được giá trị margin lớn hơn bằng cách dịch mặt $\mathbf{m}_1^{\mathrm{T}}\mathbf{x} + m_2 = 0$ về phía tập đỏ. Nhưng mà mặt $\mathbf{m}_1^{\mathrm{T}}\mathbf{x} + m_2 = 0$ đã là một phương án tối ưu rồi, vậy nên không thể tồn tại bất kỳ mặt nào khác có thể cho giá trị margin lớn hơn nữa.

+) Từ đó, ta kết luận rằng giả sử $d_1 \neq d_2$ là vô lý, không thể xảy ra, hay là khi tìm được kết quả tối ưu của bài toán (2) thì $d_1$ luôn bằng $d_2$.

+) Từ kết luận trên, ta đưa ra nhận xét: khi giải bài toán tối ưu (2) chính là việc mình đang đi tìm các *hyperplane* cách đều 2 tập với một khoảng cách lớn nhất, tức là hoàn toán giống với mục tiêu của thuật toán SVM → đpcm.

Một nhận xét quan trọng nhất của bài toán tối ưu (2): nếu ta thay vector hệ số $\mathbf{w}$ bởi $k\mathbf{w}$ và $b$ bởi $kb$ trong đó $k$ là một hằng số dương thì khoảng cách từ từng điểm đến *hyperplane* phân loại không đổi, tức *margin* không đổi. Dựa trên tính chất này, ta có thể giả sử:

$$y_n\big(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b\big) = 1$$

với những điểm gần *hyperplane* phân loại nhất.

Như vậy, bài toán tối ưu (2) sẽ trở thành:

$$\begin{aligned} (\mathbf{w}, b) &= \arg\max_{\mathbf{w},b} \left\{ \frac{1}{\|\mathbf{w}\|_2} \right\} \\ \text{vđk:} \quad & y_n(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b) \geq 1, \quad \forall n = 1, 2, \ldots, N \end{aligned} \tag{3}$$

Biến đổi đơn giản một chút, nó sẽ trở thành:

$$\begin{aligned} (\mathbf{w}, b) &= \arg\min_{\mathbf{w},b} \left\{ \|\mathbf{w}\|_2^2 \right\} = \mathbf{w}^{\mathrm{T}}\mathbf{I}\mathbf{w} \\ \text{vđk:} \quad & 1 - y_n(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b) \leq 0, \quad \forall n = 1, 2, \ldots, N \end{aligned} \tag{4}$$

Nhìn vào bài toán tối ưu (4), ta thấy hàm mục tiêu có *Hessian* là $\mathbf{I}$, xác định dương, nên đây là hàm lồi chặt, các hàm *constraint* là các hàm tuyến tính nên đây cũng là các hàm lồi, do đó bài toán tối ưu (4) là bài toán tối ưu hàm lồi, hay cụ thể là *Quadratic Programming*. Đến đây thì ta có thể sử dụng thư viên tối ưu lồi *CVXOPT* trong *Python* để giải.

## 2.2 Soft margin SVM

### 2.2.1 Ý tưởng của Soft margin

Trong thực tế, hầu hết dữ liệu (bao gồm cả dữ liệu thư rác) thường bị nhiễu hoặc chồng lấn lên nhau, khiến việc tìm một siêu phẳng phân tách hoàn toàn là không khả thi. Soft Margin SVM được ra đời để giải quyết vấn đề này.

Soft Margin SVM cho phép mô hình **mắc một số sai số** nhất định. Thay vì bắt buộc tất cả các điểm phải nằm đúng phía và ngoài vùng lề, thuật toán chấp nhận một vài điểm có thể nằm trong lề hoặc thậm chí sai phía của siêu phẳng để đổi lấy một biên phân loại có tính tổng quát hóa cao hơn.

Việc này giúp mô hình tránh được hiện tượng **Overfitting** (quá khớp) khi dữ liệu có chứa nhiễu hoặc các điểm dị biệt (outliers).

### 2.2.2 Mô hình toán học

Để toán học hóa việc "chấp nhận sai số", SVM giới thiệu thêm các **biến lỏng (slack variables)** ký hiệu là $\xi_i$ ($\xi_i \geq 0$).

- Nếu $\xi_i = 0$: Điểm dữ liệu được phân loại đúng và nằm ngoài lề.

- Nếu $0 < \xi_i \leq 1$: Điểm dữ liệu nằm trong vùng lề nhưng vẫn ở đúng phía phân loại.
- Nếu $\xi_i > 1$: Điểm dữ liệu nằm sai phía của siêu phẳng (sai số phân loại).

Lúc này, điều kiện ràng buộc trở thành:

$$y_i(\mathbf{w}^{\mathrm{T}}\mathbf{x}_i + b) \geq 1 - \xi_i, \forall i = 1, \ldots, n$$

Hàm mục tiêu của bài toán tối ưu sẽ bao gồm cả việc tối đa hóa lề và giảm thiểu tổng sai số:

$$\min_{\mathbf{w},b,\xi} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n} \xi_i$$

Trong đó $C$ là tham số điều chỉnh (regularization parameter):

- $C$ **lớn**: Phạt nặng các sai số, mô hình sẽ cố gắng phân loại đúng mọi điểm dữ liệu (tiệm cận về Hard Margin).
- $C$ **nhỏ**: Chấp nhận nhiều sai số hơn để ưu tiên một lề rộng, giúp mô hình ổn định hơn với dữ liệu mới.

# 3 Các bước tiến hành

## 3.1 Dataset

Dự án sử dụng bộ dữ liệu bao gồm 5574 tin nhắn SMS tiếng Anh đã được gán nhãn. Toàn bộ các tin nhắn ở dưới dạng văn bản bản tự nhiên, cần qua quy trình xử lý để trích các chuẩn cần thiết. Link dataset trên Kaggle

## 3.2 Quy trình xử lý dữ liệu thô

### 3.2.1 Làm sạch dữ liệu (Data Cleaning)

**Gán nhãn (Label Encoding):** Chuyển nhãn văn bản thành dạng số (0 và 1).

**Kiểm tra trùng lặp:** Loại bỏ các bản ghi lặp lại để đảm bảo tính khách quan.

### 3.2.2 Tiền xử lý văn bản (Text Preprocessing)

**Chuyển về chữ thường:** Đưa tất cả các ký tự về dạng chữ thường để tránh việc mô hình coi "Spam" và "spam" là hai từ khác nhau.

**Loại bỏ ký tự đặc biệt và dấu câu:** Loại bỏ các ký hiệu không mang giá trị ngữ nghĩa trong việc phân loại như @, #, $, %, ...

**Loại bỏ từ dừng:** Loại bỏ các từ xuất hiện phổ biến nhưng không giúp phân biệt nội dung (ví dụ: is, the, and, ...).

**Chuẩn hóa từ về dạng gốc:** Chuyển đổi các từ đã được chia động từ hoặc các biến thể về dạng gốc.Ví dụ: "playing", "played", "plays" → "play". Việc này giúp giảm kích thước từ điển và tập trung vào ý nghĩa cốt lõi của từ.

### 3.2.3 Trích xuất đặc trưng

Sau khi tiền xử lý, tôi sử dụng kỹ thuật TF-IDF (Term Frequency - Inverse Document Frequency) để biến văn bản thành vector. TF-IDF giúp đánh giá tầm quan trọng của từ dựa trên tần suất xuất hiện trong văn bản và độ hiếm trong toàn bộ tập dữ liệu, giúp mô hình tập trung vào các từ khóa mang tính phân loại cao.

## 3.3 Các chỉ số đánh giá (Metrics)

Hiệu năng của mô hình được đánh giá qua 4 chỉ số: **Accuracy, Precision, Recall** và **F1-score**.

Ưu tiên chỉ số **Precision**: Trong bài toán phát hiện thư rác, tôi đặc biệt ưu tiên **Precision**. Lý do là vì việc phân loại nhầm một thư quan trọng (Ham) thành thư rác (Spam) gây ra hậu quả nghiêm trọng hơn so với việc bỏ sót một thư rác. Một hệ thống tốt cần đảm bảo rằng những email bị chặn phải thực sự là thư rác để tránh làm gián đoạn công việc của người dùng và cũng như hạn chế tối đa việc phân loại nhầm thư Ham thành Spam.

## 3.4 Triển khai thuật toán

### 3.4.1 Công cụ và môi trường triển khai

Toàn bộ dự án được triển khai bằng ngôn ngữ lập trình **Python** trên môi trường **Jupyter Lab**. Thư viện cốt lõi được sử dụng là **scikit-learn (sklearn)** — một thư viện mạnh mẽ cung cấp các công cụ nhất quán để huấn luyện, đánh giá và tinh chỉnh các mô hình học máy.

### 3.4.2 Chia tập dữ liệu

Trước khi huấn luyện, dữ liệu được chia thành hai phần bằng hàm train_test_split:

- Tập huấn luyện (Training set): Chiếm 75% dữ liệu, dùng để mô hình học các đặc trưng của thư.
- Tập kiểm tra (Testing set): Chiếm 25% dữ liệu, dùng để đánh giá khách quan khả năng dự đoán của mô hình trên dữ liệu mới.

### 3.4.3 Cài đặt mô hình các loại

Trong Scikit-learn, mỗi thuật toán được triển khai thông qua các lớp (classes) riêng biệt với quy trình chung: Khởi tạo → Huấn luyện (fit) → Dự đoán (predict).

**Mô hình Support Vector Machine (SVM)**

Sử dụng lớp sklearn.svm.SVC (Support Vector Classification).

- Kernel: Sử dụng kernel='linear'
- Tham số $C$: Được tinh chỉnh để cân bằng giữa độ rộng của lề và sai số huấn luyện.

**Mô hình Naive Bayes**

Sử dụng lớp sklearn.naive_bayes.MultinomialNB. Đây là biến thể phù hợp nhất cho bài toán phân loại văn bản vì nó làm việc dựa trên tần suất xuất hiện của các từ (đặc biệt hiệu quả với vector TF-IDF).

**Mô hình k-Nearest Neighbors (kNN)**

Sử dụng lớp sklearn.neighbors.KNeighborsClassifier. Mô hình phân loại dựa trên khoảng cách Euclidean giữa các vector văn bản. Tham số số láng giềng $k$ được lựa chọn thông qua thực nghiệm để tránh hiện tượng nhiễu.

**Mô hình Logistic Regression**

Sử dụng lớp sklearn.linear_model.LogisticRegression. Tôi sử dụng hàm tối ưu hóa mặc định để tìm ra ranh giới phân tách xác suất giữa hai lớp Ham và Spam.

# 4 Kết quả

## 4.1 Kết quả đối với từng thuật toán

### SVM

Accuracy score of Soft marign SVM for training set: 0.9990428332136875

Accuracy score of Soft marign SVM for testing set: 0.9734386216798278

---

Precision score of Soft marign SVM for training set: 0.9981751824817519

Precision score of Soft marign SVM for testing set: 0.925531914893617

---

Recall score of Soft marign SVM for training set: 0.9945454545454545

Recall score of Soft marign SVM for testing set: 0.883248730964467

---

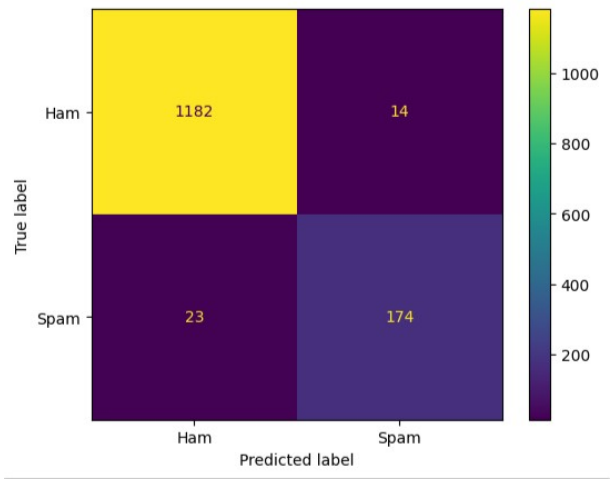
F1 score of Soft marign SVM for training set: 0.9963570127504554

F1 score of Soft marign SVM for testing set: 0.9038961038961039

---

Confusion matrix of Soft margin SVM for testing set:

[[1182 14] [ 23 174]]




Hình 2: Confusion matrix đối với thuật toán SVM.

**Naive Bayes**

Accuracy score of NB for training set: 0.9806173725771715

Accuracy score of NB for testing set: 0.9734386216798278

---

Precision score of NB for training set: 0.9978768577494692

Precision score of NB for testing set: 0.9878048780487805

---

Recall score of NB for training set: 0.8545454545454545

Recall score of NB for testing set: 0.8223350253807107

---

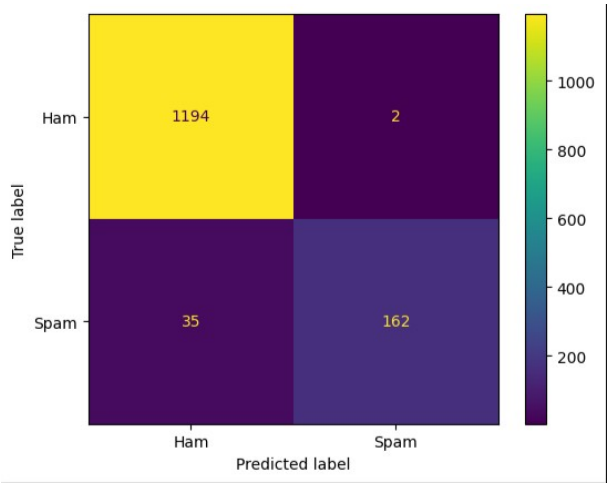
F1 score of NB for training set: 0.920666013712047

F1 score of NB for testing set: 0.8975069252077562

---

Confusion matrix of Naive Bayes for testing set:

[[1194 2] [ 35 162]]




Hình 3: Confusion matrix đối với thuật toán Naive Bayes.

**Logistics Regression**

Accuracy score of LR for training set: 0.9691313711414213

Accuracy score of LR for testing set: 0.964824120603015

---

Precision score of LR for training set: 0.9839080459770115

Precision score of LR for testing set: 0.9743589743589743

---

Recall score of LR for training set: 0.7781818181818182

Recall score of LR for testing set: 0.7715736040609137

---

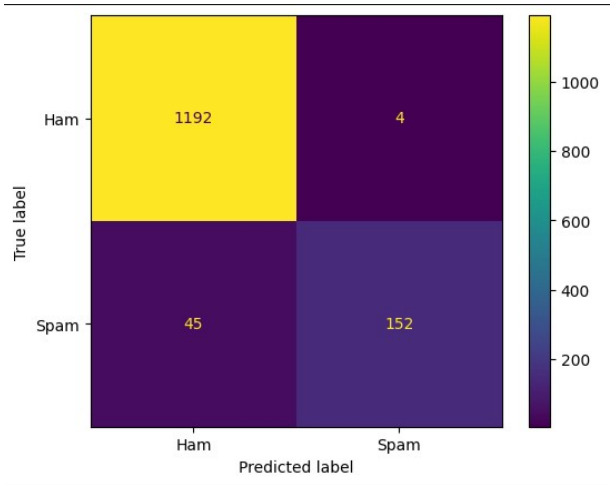
F1 score of LR for training set: 0.8690355329949239

F1 score of LR for testing set: 0.8611898016997167

---

Confusion matrix of Logistic Regression for testing set:

[[1192 4] [ 45 152]]




Hình 4: Confusion matrix đối với thuật toán Logistics Regression.

**kNN, n_neighbors = 7**

Accuracy score of kNN for training set: 0.9011725293132329

Accuracy score of kNN for testing set: 0.886575735821967

---

Precision score of kNN for training set: 1.0

Precision score of kNN for testing set: 0.975609756097561

---

Recall score of kNN for training set: 0.24909090909090909

Recall score of kNN for testing set: 0.20304568527918782

---

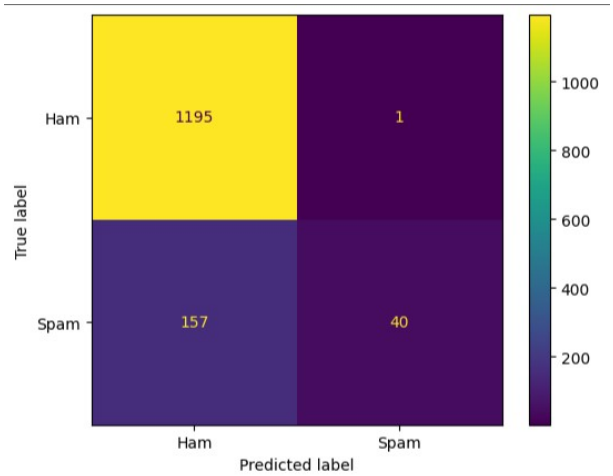
F1 score of kNN for training set: 0.3988355167394469

F1 score of kNN for testing set: 0.33613445378151263

---

Confusion matrix of kNN for testing set:

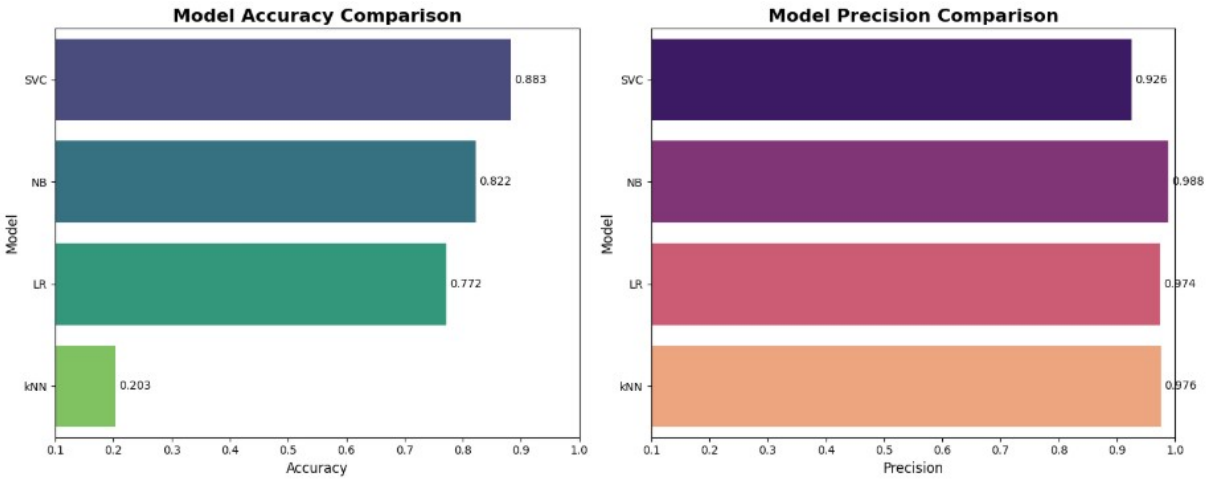
[[1195 1] [ 157 40]]




Hình 5: Confusion matrix đối với thuật toán kNN.

## 4.2 So sánh kết quả giữa các thuật toán

Biểu đồ so sánh kết quả giữa các thuật toán:




Hình 6: Biểu đồ so sánh kết quả giữa các thuật toán.

Như đã đề cập ở trên, khi dự đoán một email spam hay ham, ta sẽ phải ưu tiên các mô hình có *precison_score* cao, bởi vì cái giá phải trả cho việc phân loại ham thành spam sẽ khá đắt. Ví dụ: nếu mô hình phân loại một email thông báo trúng tuyển thành spam thì ảnh hưởng đến cả sự nghiệp của user. Vậy nên, dựa vào kết quả ta thấy mô hình Naive Bayes có hiệu quả phân loại tốt nhất, và sẽ làm việc tốt hơn mô hình Soft margin SVM.